Plum Health Insurance

AI Claims Processing System

Architecture Document

Assignment Submission - AI Engineer Role

April 2026

1. System Overview

A claims reviewer today reads each uploaded document, checks it against the policy, and writes a decision. This system does that same job automatically. It reads the documents, applies the rules, and produces a decision with a full explanation of why.

Core Problem

Claims review is slow and manual today. A reviewer reads each document, looks up the policy, and writes a decision. This system does the same thing for every claim automatically, and every decision comes with a full trace of why it was made.

1.1 What the System Does

- Accepts a claim submission - member details, treatment type, claimed amount, uploaded PDFs or images
- Verifies documents immediately - wrong type, unreadable, or cross-patient documents are caught before any processing
- Extracts structured information - GPT-4o vision reads messy real-world Indian medical documents
- Applies policy rules - waiting periods, exclusions, pre-authorization, per-claim limits, network discounts, co-pay
- Detects fraud signals - unusual same-day claim patterns flagged for manual review
- Produces an explainable decision - APPROVED, PARTIAL, REJECTED, or MANUAL_REVIEW with full audit trace

1.2 Decision Types

Decision	Meaning	Example
APPROVED	Claim passes all checks, amount calculated	TC004 - viral fever consultation, ₹1,350 approved
PARTIAL	Some line items covered, others excluded	TC006 - root canal approved, teeth whitening excluded
REJECTED	Policy rule prevents approval	TC005 - diabetes within 90-day waiting period
MANUAL_REVIEW	Fraud signals detected, human must decide	TC009 - 4 claims on same day

2. Architecture

2.1 System Components

The system is built as a sequential multi-agent pipeline orchestrated by LangGraph. Each agent is a discrete node in the graph with a single responsibility. State flows between nodes via a typed dictionary.

Layer	Component	Technology	Responsibility
API	Claims gateway	FastAPI	Request validation, routing, CORS
API	Upload endpoint	FastAPI multipart	Real file ingestion, type detection
Orchestration	Pipeline	LangGraph	Agent sequencing, error isolation, state
Agent 1	Document Verifier	Pure Python	Early exit on wrong/unreadable/mismatched docs
Agent 2	Document Parser	GPT-4o vision	Extract patient, diagnosis, amounts from PDFs
Agent 3	Policy Evaluator	Pure Python	Apply all policy rules from policy_terms.json
Agent 4	Fraud Detector	Pure Python	Flag unusual claim patterns
Agent 5	Decision Synthesizer	Pure Python	Combine agent outputs into final decision
Core	Policy Engine	Pure Python	Stateless functions, reads policy_terms.json
Core	File Handler	pymupdf + Pillow	PDF→JPEG conversion, blur detection, base64
UI	React Frontend	React 19 + Vite	Claim submission, decision review, trace viewer

2.2 Request Flow

Every claim goes through the same five agents in order. Each agent gets the previous agent's output as input. If an agent fails - say the GPT-4o call times out - the pipeline records the failure and keeps going. It never crashes.

Pipeline Flow

Member submits claim + documents ↓ POST /api/v1/claims/upload ↓ file_handler converts PDFs to JPEG via pymupdf Agent 1: Document Verifier → If errors: return 422 with specific error messages (TC001, TC002, TC003) → If clean: continue ↓ Agent 2: Document Parser (GPT-4o vision) → Extracts: patient name, diagnosis, doctor, line items, amounts ↓ Agent 3: Policy Evaluator (pure Python) → Checks: member, waiting period,

pre-auth, per-claim limit, exclusions, line items → Calculates: network discount → copay → approved amount ↓ Agent 4: Fraud Detector (pure Python) → Checks: same-day claim count, high-value thresholds ↓ Agent 5: Decision Synthesizer (pure Python) → Produces: decision + amount + confidence + full trace ↓ JSON response with complete audit trail

2.3 Data Model

All agents communicate through Pydantic v2 models. The models act as the contract between components.

Model	Purpose	Key Fields
ClaimSubmission	Input to the pipeline	member_id, claim_category, treatment_date, claimed_amount, documents[]
DocumentInfo	One uploaded document	file_id, actual_type, quality, content (structured), file_bytes (raw)
ClaimDecision	Final output	decision, approved_amount, confidence_score, message, trace[], component_failures[]
TraceStep	One agent's audit record	agent, status, detail, confidence_delta, data
LineItemDecision	Per line item result	description, claimed_amount, approved_amount, reason
DocumentVerificationError	Early exit error	error_code, message, file_id, uploaded_type, required_types

3. Design Decisions

3.1 Why LangGraph for Orchestration

Decision: Use LangGraph to orchestrate the 5-agent pipeline.

Why: Each agent is a node in the graph. Every node is wrapped in try/except. If the parser fails - API timeout, bad response, anything - LangGraph logs the failure, drops the confidence score, and moves to the next node. The claim still gets a decision. It never crashes.

This covers TC011 (graceful degradation) and the assignment requirement that component failure must not crash the pipeline.

Considered	Rejected Because
Single function calling all agents	One failure crashes everything - no isolation
Celery task queue	Overkill for synchronous pipeline; adds Redis dependency
LangChain LCEL	Less flexible for stateful pipelines with conditional routing
LangGraph (chosen)	Native graph + state + error isolation + conditional edges

3.2 LLM Only for Document Extraction

Decision: Use GPT-4o vision only in Agent 2 (document parser). All other agents use pure Python.

Why: Waiting period checks are date math. Exclusion checks are keyword lookups. These do not need an LLM. Putting policy logic inside a prompt makes it non-deterministic and hard to test. GPT-4o is used for the one thing it is actually good at here - reading a blurry handwritten prescription and pulling out the diagnosis.

> **Key Principle**
> LLM = reading documents (unstructured → structured) Pure Python = applying rules (structured data → decision) This keeps decisions testable and auditable.

3.3 Policy Rules Read from JSON, Never Hardcoded

Decision: All policy values - waiting periods, sub-limits, co-pay percentages, exclusions, network hospitals - are read from policy_terms.json at runtime.

Why: If co-pay percentages or waiting periods were hardcoded, every policy update would need a code change and a deployment. Reading from JSON means a policy change is just a file update. Every function in policy_engine.py takes a value from that file and applies it - no side effects, easy to test.

3.4 pymupdf for PDF Processing

Decision: Use pymupdf (fitz) to convert PDF pages to JPEG images before sending to GPT-4o vision.

Why: GPT-4o only accepts images, not PDFs. The first approach used pdf2image, which needs poppler installed on the system. Poppler does not work on Windows without manual setup, and breaks on most PaaS platforms. pymupdf installs with pip, has no system dependencies, and works the same on Windows, Linux, and any cloud host.

Approach	System Deps	Windows	Cloud	Status
pdf2image + poppler	Yes (poppler)	Manual install	Breaks on PaaS	Rejected
Base64 PDF to GPT-4o	None	Works	Works	Rejected (invalid MIME)
pymupdf (chosen)	None	Works	Works	Chosen

4. Observability

4.1 Audit Trail Design

Every claim decision is fully reconstructible from the trace. Each agent writes a TraceStep that records exactly what it checked, what it found, and what effect it had on confidence.

TraceStep Field	Type	Purpose
agent	string	Which agent produced this step
status	SUCCESS | FAILED | SKIPPED	Whether the agent completed normally
detail	string	Human-readable summary of what the agent did and found
confidence_delta	float	How much this step changed the overall confidence score
data	dict	Structured data for downstream inspection (checks_run, rejection_reasons, etc.)

4.2 Confidence Score

The final confidence score starts at 1.0 and is reduced by each agent's confidence_delta. This gives an at-a-glance quality signal:

- Parser fails (e.g. API timeout): −0.4 confidence
- Fraud signals detected: −0.2 confidence
- Component failure: −0.15 per failure
- Hard policy rejections (exclusion, waiting period): confidence stays high - the rule is certain

4.3 Reconstructing a Decision

Given only the JSON response from the API, a reviewer can answer every question about why a decision was made:

- What documents were uploaded and were they valid? → document_verifier trace step
- What did the system read from those documents? → document_parser trace step + data field
- Which policy checks passed and which failed? → policy_evaluator trace step with checks_run list
- Were there fraud signals? → fraud_detector trace step

- How was the approved amount calculated? → calculation_breakdown field
- Did any component fail during processing? → component_failures field

5. Limitations & Scale

5.1 Current Limitations

Limitation	Impact	Root Cause
No persistent storage	Claims not stored - each request is stateless	No database - SQLite or Postgres needed
YTD limits not enforced	Annual sub-limits checked informally, not tracked	No claims history store
Single-document PDF only	Multi-page prescriptions may lose context	Each page sent separately to GPT-4o
No async processing	GPT-4o call blocks the request thread	Synchronous FastAPI endpoint
No auth/auth	Any caller can submit claims	No JWT or API key validation
OCR quality dependent on GPT-4o	Very low quality images may fail extraction	No fallback OCR engine

5.2 Design at 10x Load

At 100 concurrent submissions these things would need to change:

Async Processing

The GPT-4o vision call is the slow part - it takes 3-8 seconds per document. At 10x load, synchronous processing would queue requests. The fix is to make the pipeline async with FastAPI background tasks or Celery:

- Accept claim → return claim_id immediately (202 Accepted)
- Process pipeline in background worker
- Client polls GET /claims/{claim_id} for result

Persistent Storage

Claims, decisions, and traces need to be stored for audit purposes and for enforcing annual limits:

- PostgreSQL for claims, decisions, member history
- Redis for same-day claim count lookups (fraud detection)
- S3 / blob storage for uploaded documents

Rate Limiting & Caching

- Rate limit the OpenAI API calls - use a token bucket per member

- Cache policy_terms.json in Redis - avoid file read on every request
- Cache member lookup results - member roster changes rarely

Horizontal Scaling

- FastAPI + Uvicorn workers scale horizontally behind a load balancer
- LangGraph pipeline is stateless - no shared state between requests
- Each worker runs the full pipeline independently

Component	Current	At 10x
Storage	None (stateless)	PostgreSQL + S3 + Redis
Processing	Sync, blocks thread	Async + Celery workers
Scaling	Single process	Horizontal - N Uvicorn workers
Policy data	File read per request	Redis cache, TTL 1 hour
Auth	None	JWT + API key per member portal
Monitoring	Print logs	Structured logging + Datadog / Grafana

6. Test Coverage

6.1 Test Suite Overview

Test File	Tests	Coverage
test_verifier.py	5	TC001, TC002, TC003 + 2 happy path document checks
test_pipeline.py	9	TC004–TC012 - full pipeline end to end
test_file_handler.py	15	File type detection, blur scoring, PDF/image processing, vision content builder

Total: 29 tests, all passing. Zero external API calls in tests - all 12 claim decision tests use structured content dicts, not real LLM calls.

6.2 Test Design Principles

- Policy engine functions are pure - every function has deterministic input/output, easily unit tested
- Pipeline tests use structured content - no LLM calls, no external dependencies, fast and reliable
- File handler tests cover real PDFs from sample_docs - actual pymupdf conversion verified
- TC011 (graceful degradation) is tested by setting simulate_component_failure=True on the submission

7. Technology Stack

Layer	Technology	Version	Why
API	FastAPI	0.136	Async, Pydantic native, auto Swagger docs
Agent orchestration	LangGraph	1.1.8	Graph-based pipeline with error isolation
AI	OpenAI GPT-4o	Latest	Best vision model for messy document extraction
Data validation	Pydantic v2	2.13	Fast, typed, agent contract enforcement
PDF processing	pymupdf	1.27	Pure Python, no system dependencies, fast rendering
Image processing	Pillow	12.1	Blur detection for phone photos
Mock documents	fpdf2	2.8	Pure Python PDF generation for test fixtures

Layer	Technology	Version	Why
Frontend	React 19 + Vite	Latest	Fast dev experience, CSS Modules for scoping
Testing	pytest	9.0	Clean, readable, parametrize-friendly

8. What I Would Do Differently

8.1 One Decision I'm Proud Of

Keeping the LLM out of policy decisions. GPT-4o is only called in the document parser. Everything after that is pure Python. I can write a pytest test for every rejection reason and know it will always produce the same result. If policy logic lived in a prompt, I could not do that.

It also means a policy change is a JSON file update, not a prompt rewrite. That matters when someone needs to change a co-pay percentage or add a new exclusion quickly.

8.2 One Decision I Would Change

The confidence score. It starts at 1.0 and each failing agent subtracts a fixed amount. A bariatric claim rejected under a hard exclusion is completely certain - but if the parser also failed, the confidence drops to 10%. The decision is certain. The document reading was just bad. I would split it into two scores: one for extraction quality, one for decision certainty.